

```

"""
Predictive Analytics Using Historical Data
-----

Task: Build a predictive model to forecast future trends using
historical sales data.

Pipeline:
1. Generate / load historical dataset
2. Clean and preprocess data
3. Feature engineer time-based features (trend + seasonality)
4. Train a regression model
5. Evaluate accuracy (MAE, RMSE, R2)
6. Visualize actual vs predicted + future forecast
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

RNG = np.random.default_rng(42)

# -----
# 1. GENERATE / LOAD HISTORICAL DATASET
# -----
# Simulates 3 years of monthly sales data with trend + seasonality + noise.
# (Swap this block with pd.read_csv("your_data.csv") to use real data.)

n_months = 36
dates = pd.date_range(start="2023-01-01", periods=n_months, freq="MS")

trend = np.linspace(200, 500, n_months) # upward trend
seasonality = 60 * np.sin(2 * np.pi * dates.month / 12) # yearly cycle
noise = RNG.normal(0, 20, n_months) # random noise

sales = trend + seasonality + noise
sales = np.round(sales, 2)

df = pd.DataFrame({"date": dates, "sales": sales})

```

```

# Inject a few missing values & an outlier to simulate real messy data
df.loc[5, "sales"] = np.nan
df.loc[20, "sales"] = np.nan
df.loc[15, "sales"] = df["sales"].max() * 2.5 # outlier

df.to_csv("/home/claude/project/historical_sales.csv", index=False)
print("Raw data saved. Shape:", df.shape)
print(df.head())

# -----
# 2. CLEAN & PREPROCESS
# -----
df_clean = df.copy()

# Fill missing values via linear interpolation
df_clean["sales"] = df_clean["sales"].interpolate(method="linear")

# Cap outliers using IQR method
q1, q3 = df_clean["sales"].quantile([0.25, 0.75])
iqr = q3 - q1
upper_bound = q3 + 1.5 * iqr
lower_bound = q1 - 1.5 * iqr
df_clean["sales"] = df_clean["sales"].clip(lower_bound, upper_bound)

print("\nMissing values after cleaning:", df_clean["sales"].isna().sum())
print("Outlier cap applied at:", round(upper_bound, 2))

# -----
# 3. FEATURE ENGINEERING (time-based regression features)
# -----
df_clean["t"] = np.arange(len(df_clean)) # linear trend index
df_clean["month"] = df_clean["date"].dt.month
df_clean["month_sin"] = np.sin(2 * np.pi * df_clean["month"] / 12)
df_clean["month_cos"] = np.cos(2 * np.pi * df_clean["month"] / 12)

features = ["t", "month_sin", "month_cos"]
X = df_clean[features]
y = df_clean["sales"]

# -----
# 4. TRAIN / TEST SPLIT + MODEL TRAINING
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False # keep chronological order
)

```

```

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42),
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    results[name] = {
        "model": model,
        "preds": preds,
        "MAE": mean_absolute_error(y_test, preds),
        "RMSE": np.sqrt(mean_squared_error(y_test, preds)),
        "R2": r2_score(y_test, preds),
    }

# -----
# 5. EVALUATE
# -----
print("\n--- Model Evaluation (on held-out test months) ---")
for name, r in results.items():
    print(f"{name:20s} | MAE: {r['MAE']:.2f} | RMSE: {r['RMSE']:.2f} | R2: {r['R2']:.2f}")

best_name = min(results, key=lambda n: results[n]["RMSE"])
best_model = results[best_name]["model"]
print(f"\nBest model: {best_name}")

# -----
# 6. FORECAST FUTURE (next 6 months)
# -----
future_periods = 6
last_t = df_clean["t"].max()
future_dates = pd.date_range(
    start=df_clean["date"].max() + pd.offsets.MonthBegin(1),
    periods=future_periods, freq="MS"
)

future_df = pd.DataFrame({"date": future_dates})
future_df["t"] = np.arange(last_t + 1, last_t + 1 + future_periods)
future_df["month"] = future_df["date"].dt.month
future_df["month_sin"] = np.sin(2 * np.pi * future_df["month"] / 12)
future_df["month_cos"] = np.cos(2 * np.pi * future_df["month"] / 12)

future_df["forecast"] = best_model.predict(future_df[features])

print("\n--- 6-Month Forecast ---")
print(future_df[["date", "forecast"]].round(2).to_string(index=False))

```

```

future_df[["date", "forecast"]].to_csv(
    "/home/claude/project/forecast_output.csv", index=False
)

# -----
# 7. VISUALIZATION
# -----

fig, axes = plt.subplots(2, 1, figsize=(11, 9))

# Plot 1: Historical actual vs full-range model fit + forecast
axes[0].plot(df_clean["date"], df_clean["sales"], "o-", label="Historical (actual
full_fit = best_model.predict(df_clean[features])
axes[0].plot(df_clean["date"], full_fit, "--", label=f"{best_name} fit", color="#
axes[0].plot(future_df["date"], future_df["forecast"], "o--", label="Forecast (ne
axes[0].axvline(df_clean["date"].max(), color="gray", linestyle=":", linewidth=1)
axes[0].set_title("Historical Sales, Model Fit & 6-Month Forecast")
axes[0].set_xlabel("Date")
axes[0].set_ylabel("Sales")
axes[0].legend()
axes[0].grid(alpha=0.3)

# Plot 2: Test-set actual vs predicted (accuracy check)
test_dates = df_clean["date"].iloc[y_test.index]
axes[1].plot(test_dates, y_test.values, "o-", label="Actual", color="#2563eb")
for name, r in results.items():
    axes[1].plot(test_dates, r["preds"], "o--", label=f"{name} predicted")
axes[1].set_title("Model Accuracy: Actual vs Predicted (test months)")
axes[1].set_xlabel("Date")
axes[1].set_ylabel("Sales")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("/home/claude/project/forecast_chart.png", dpi=150)
print("\nChart saved to forecast_chart.png")

```